```
#ifndef BST_h
#define BST_h

template <typename T>
class BinarySearchTree{
     private:
        T key;
        //pointers:
        BinarySearchTree<T>* left;
        BinarySearchTree<T>* right;


     public:
     //constructor:
     BinarySearchTree<T>(T value);
     void insert (T value);

    //triversing order:
    void preOrder ();
    void inOrder();
    void postOrder();



};

template <typename T>
BinarySearchTree<T>::BinarySearchTree(T value){
     key = value;
     left = nullptr;
     right = nullptr;
}

template <typename T>
void BinarySearchTree<T>::insert(T value){
   if(value < key)
   //do something:
   if (left == nullptr){
        left = new BinarySearchTree<T>(value);
   } else {
        left -> insert(value);
   } else if (value > key ){

    if(right == nullptr){
        right = new BinarySearchTree<T>(value);
    } else{
        right -> insert(value);
    }
   }
}


   template <typename T>
   void BinarySearchTree<T>::preOrder(){
    std::cout << key << " " << std::endl;
    if(left != nullptr){
        left -> preOrder();
    }
    if (right != nullptr){
```

```
        right -> preOrder();
    }

}


    template <typename T>
   void BinarySearchTree<T>::inOrder(){
    std::cout << key << " " << std::endl;
    if(left != nullptr){
        left -> inOrder();
    }
    if (right != nullptr){
        right -> inOrder();
    }
   }

    template <typename T>
   void BinarySearchTree<T>::postOrder(){
    std::cout << key << " " << std::endl;
    if(left != nullptr){
        left -> postOrder();
    }
    if (right =! nullptr){
        right -> postOrder();
    }
   }



#endif
```